

NTI - DIGITAL IC DESIGN USING FPGA

FINAL PROJECT: UART (RX PART)

INSTRUCTOR: ENG. MOAZ KHALED

MADE BY: AHMED MOSLEH

Digital IC DESIGN USING FPGA FINAL
PROJECT
UART (RX-PART)

احمد محمد مصلح فهمي الجندي

336308

الاسم:

كود التسجيل:

Introduction:

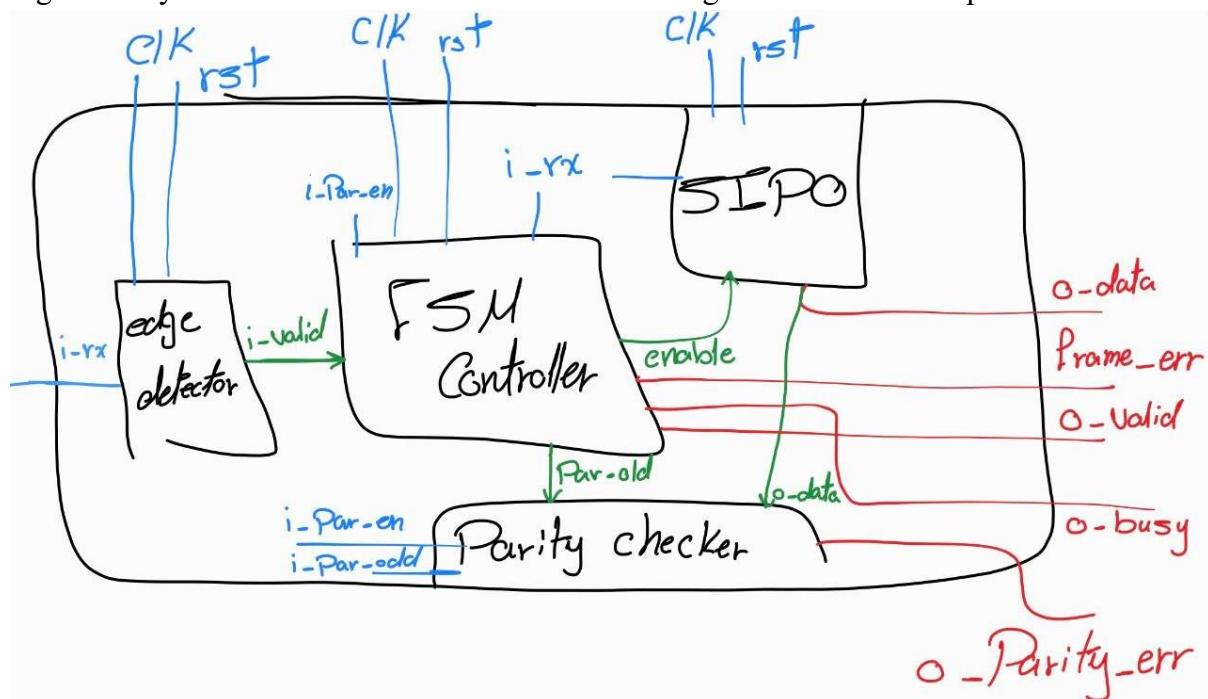
In this Project we will discuss what is a UART (especially its RX-Part), explain how it works in hardware, its implementation in **System Verilog**, and its **Schematics**.

So, what is a UART?

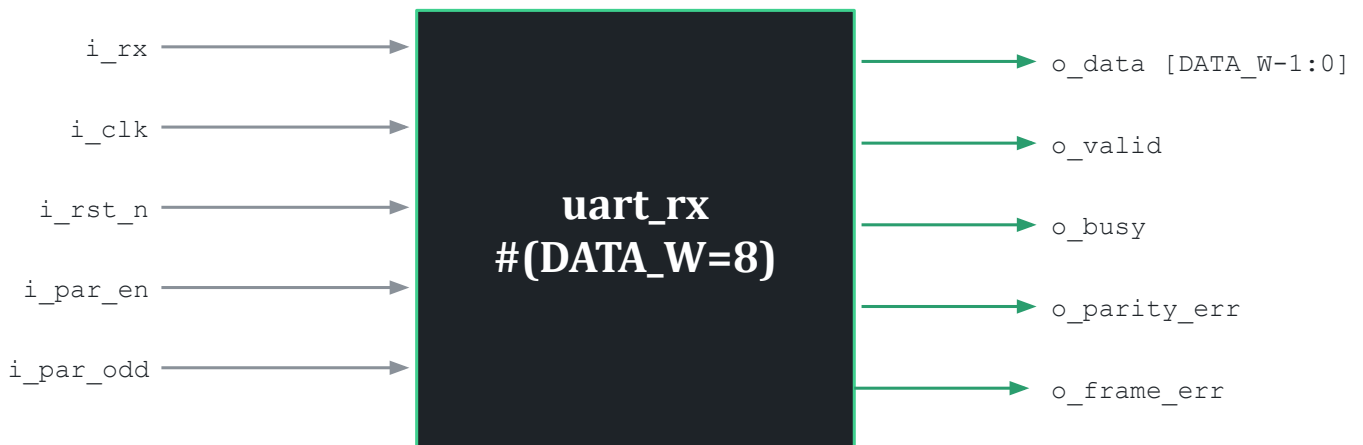
UART (Universal Asynchronous Receiver-Transmitter) is a serial communication protocol used for transmitting data between devices without a shared clock signal. It is designed to convert an 8-bit parallel input into a structured 10-bit or 11-bit serial frame ready for transmission from a Transmitter (TX) to a Receiver (RX).

How does the RX part work?

The general idea is waiting for a change in the input (from 1 to 0) that indicates the Start Bit, to start the receiving operation of the sent data, while receiving the 8-bit serial data and sending them into a SIPO (Serial IN Parallel Out) to turn it into Parallel Data, and checking the polarity of the data (if required) to compare it with the old Polarity Bit that was sent by the Transmitter and ending the transmission by sending a stop bit. All this process is organized by a Finite State Machine and filtered through a Mux for the output.



RX Frame



The ports of the RX consist of:

Input:

i_rx → This is the port that will have the 10-bit or 11-bit serial data that will go to the receiver.

i_clk → System clock.

i_rst_n → Asynchronous active-low reset signal to clear the registers.

i_par_en → Parity Enable Control (1: Enable, 0: Disable).

i_par_odd → Parity Bit Selection (0: Even Parity, 1: Odd Parity).

Output:

o_data → Parallel Data Output (Also contains Start bit, Polarity Check and Stop bit).

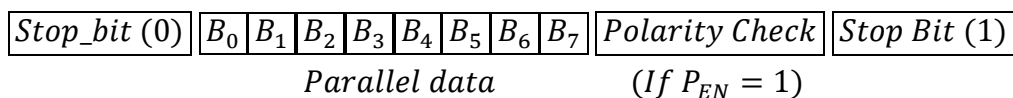
o_valid → Status signal that pulses high for one cycle holding the received data.

o_busy → Status signal that goes high (1) during transmission and low (0) otherwise.

o_parity_err → Status signal that pulses high for one cycle when **i_par_en** = 1 and the parity received disagrees with the one calculated.

o_frame_err → Status signal that pulses high for one cycle (The same cycle as **o_valid**) when the stop bit isn't 1.

The input data should be like this:



To make the RX we need Four Modules which are:

1. Negative Edge Detector
2. Finite State Machine Controller (FSM Controller)
3. Series to Parallel Shift Register (SIPO)
4. Polarity Bit Calculator

Let's talk about each module in detail.

1. **Negative Edge Detector:**

The starter of the whole operation. It detects when a signal goes from 1 to 0 (Negative Edge) and signals it to the **FSM Controller** telling it that this is the start bit, leading to starting the whole process in the receiver.

How does it work?

simply it takes the original data and stores it in a register then compares it with the inverse of the next data (by using And Gate) if the output is 1 then a Negative edge happened.

The System Verilog code:

```
module Neg_edge_detect (
input logic clk, rst, in,
output logic neg_edge_A
);

logic in_old;

always_ff @(posedge clk or negedge rst)
begin
    if (!rst)
    begin
        in_old <= 0;
    end

    else
    begin
        in_old <= in;
    end
end

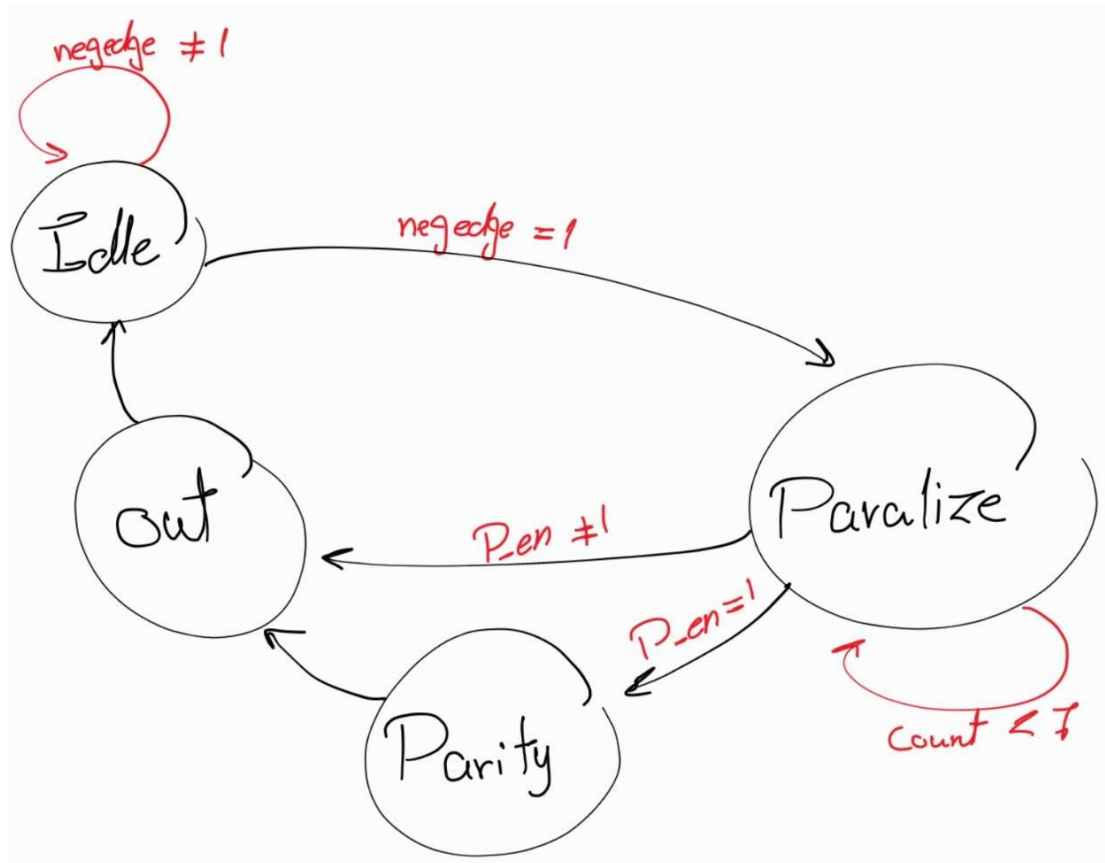
assign neg_edge_A = !in && in_old;

endmodule
```


2. Finite State Machine Controller (FSM Controller):

Also called the brain of the Top Module (RX), which is a Four-State FSM that is responsible for every step in the Top Module (RX), and sending signals to the other modules telling them when to start, and the sets the **o_busy** high. This ensures that the circuit follows a chronological order for each bit received.

FSM design:



1. **Idle State** → The System waits for the negative edge in i_{rx} .
From Idle to Paralyze → o_busy starts to signal high till the end of the process.
2. **Paralyze State (also named Load)** → The SIPO Parallelizes the data and sends it to both the parity bit calculator and the output.
After this state the next state depends on i_par_en .
$$Next\ state \begin{cases} Parity\ State, & \text{if } i_par_en = 1 \\ Stop\ State, & \text{if } i_par_en = 0 \end{cases}$$
3. **Parity State** → The FSM sends the parity bit to the Parity Bit Calculator to check the parity of the data then it compares it with the sent parity bit if it is the same then o_par_err is set to 0 otherwise it is set to 1 flagging an error has occurred.
4. **Stop State** → This is the final state where the FSM checks the Stop Bit to see whether it is 1 or 0 if it is 0 the o_frame_err pulses high.
#Note the o_frame_err is sent in the next idle state but it needs to pass on the stop state first.

The System Verilog Code:

```
module FSM_Controller_RX #(parameter DATA_W = 8) (  
    input logic i_clk, i_rst_n, i_par_en, i_valid, i_rx,  
    output logic SIPO_EN, o_busy, Par_old, o_frame_err, o_valid  
);  
  
    logic [$clog2(DATA_W):0] counter;  
    logic Parity_bit, end_state, stop_bit;  
  
    typedef enum logic [1:0] { Idle, Load, Parity_Check, Stop } states;  
  
    states current, next;  
  
    always_ff @(posedge i_clk or negedge i_rst_n)  
    begin  
        if (!i_rst_n)  
            begin  
                current <= Idle;  
                counter <= 4'd0;  
                Parity_bit <= 0;  
                end_state <= 0;  
                stop_bit <= 1;  
            end  
  
        else  
            begin  
                current <= next;  
  
                if (current == Load)  
                    begin  
                        counter <= counter + 4'd1;  
                    end  
  
                else  
                    begin  
                        counter <= 0;  
                    end  
  
                if (current == Parity_Check)  
                    begin  
                        Parity_bit <= i_rx;  
                    end  
  
                end_state <= (current == Stop);  
  
                if (current == Stop)  
                    begin  
                        stop_bit <= (i_rx != 1);  
                    end  
            end  
        end  
    end  
  
    // start(0) b0 b1 b2 b3 b4 b5 b6 b7 stop(1)  
  
    always_comb  
    begin  
        next = current;  
        SIPO_EN = 0;  
        o_busy = 0;  
        o_frame_err = 0;  
        o_valid = 0;  
        Par_old = Parity_bit;  
    end  
end
```

```

case (current)
Idle : begin

    if (i_valid)
    begin
        next = Load;
        o_busy = 1;
    end

    else
    begin
        next = Idle;
        o_busy = 0;
    end

    if (end_state)
    begin
        o_frame_err = (i_rx != 1'b1);
        o_valid = 1;
    end

end

Load : begin

    if (counter == DATA_W - 1)
    begin
        if (i_par_en)
        begin
            next = Parity_Check;
        end

        else
        begin
            next = Stop;
        end

    end

    else
    begin
        next = Load;
    end

    o_busy = 1;
    SIPO_EN = 1;

end

Parity_Check : begin

    o_busy = 1;
    next = Stop;

end

Stop : begin

    next = Idle;
    o_busy = 1;

end

default : begin

    next = Idle;

end

endcase
end
endmodule

```


3. Series To Parallel Shift Register (SIPO):

This is the part that is responsible for converting the 8-bit serial data to parallel.

The System Verilog Code:

```
module Serializer #(parameter DATA_W = 8) (  
    input logic [DATA_W - 1:0] i_data,  
    input logic i_clk, i_rst_n, start, load,  
    output logic out  
);  
  
    logic [$clog2(DATA_W) - 1:0] counter;  
    logic [DATA_W - 1:0] temi_data;  
    logic busy;  
  
    always_ff @(posedge i_clk or negedge i_rst_n)  
    begin  
        if (!i_rst_n)  
            begin  
                temi_data <= 0;  
                busy <= 0;  
                out <= 0;  
                counter <= 3'd0;  
            end  
  
            else if (load && !busy)  
            begin  
                temi_data <= i_data;  
                counter <= 3'd0;  
            end  
  
            else if (start && !busy)  
            begin  
                busy <= 1;  
                counter <= 3'd1;  
                out <= temi_data[0];  
            end  
  
            else if (busy)  
            begin  
                out <= temi_data[counter];  
                counter <= counter + 3'd1;  
  
                if (counter == DATA_W - 1)  
                begin  
                    busy <= 0;  
                end  
            end  
        end  
    end  
endmodule
```

4. Parity Bit Calculator:

This module is responsible for checking the polarity of the Parallel data (If i_par_en = 1)

How it works?

It simply uses XOR Gate as a comparator, since XOR outputs 1 if the number of high bits is odd and outputs 0 otherwise.

So we XOR every bit in the 8-bit data and if the i_par_odd is 1 we send the current output otherwise we flip the output and send it.

What is it used for?

It is used as a data error check where it is present in both the TX and the RX. Both check the parity of the data and compare it between them, if there is a difference between them then there is a problem that needs to be fixed.

The System Verilog Code:

```
module Parity_Bit_Calculator_RX #(parameter DATA_W = 8) (  
    input logic [DATA_W - 1:0] i_data,  
    input logic i_par_odd, i_par_en, par_old,  
    output logic o_parity_err  
);  
  
    logic compare, P_out;  
  
    always_comb  
    begin  
        compare = ^i_data;  
        if (i_par_en)  
        begin  
            if (i_par_odd)  
            begin  
                P_out = compare;  
            end  
  
            else  
            begin  
                P_out = !compare;  
            end  
  
            o_parity_err = P_out ^ par_old;  
        end  
  
        else  
        begin  
            o_parity_err = 0;  
            P_out = 0;  
        end  
    end  
end  
endmodule
```

Now let's combine all of them and put them in our Top Module the TX. All we need to do is just instantiate the four previous modules and give them the required inputs and output.

The System Verilog Code:

```
module uart_rx #(parameter DATA_W = 8) (  
    input logic i_rx, i_clk, i_rst_n, i_par_en, i_par_odd,  
    output logic [DATA_W - 1 : 0] o_data,  
    output logic o_valid, o_busy, o_parity_err, o_frame_err  
);  
  
    logic valid_zero, SIPO_EN, Par_old;  
    logic [DATA_W - 1 : 0] Data;  
  
    Neg_edge_detect STOP_BIT_DETECT ( .clk(i_clk), .rst(i_rst_n), .in(i_rx), .neg_edge_A(valid_zero) );
```

```

FSM_Controller_RX #(.DATA_W(DATA_W)) FSMC_RX ( .i_clk(i_clk), .i_rst_n(i_rst_n), .i_par_en(i_par_en),
.i_valid(valid_zero), .i_rx(i_rx), .SIPO_EN(SIPO_EN),

.o_busy(o_busy), .Par_old(Par_old), .o_frame_err(o_frame_err), .o_valid(o_valid) );

Parity_Bit_Calculator_RX #(.DATA_W(DATA_W)) PC ( .i_data(Data), .i_par_odd(i_par_odd),
.i_par_en(i_par_en), .par_old(Par_old), .o_parity_err(o_parity_err) );

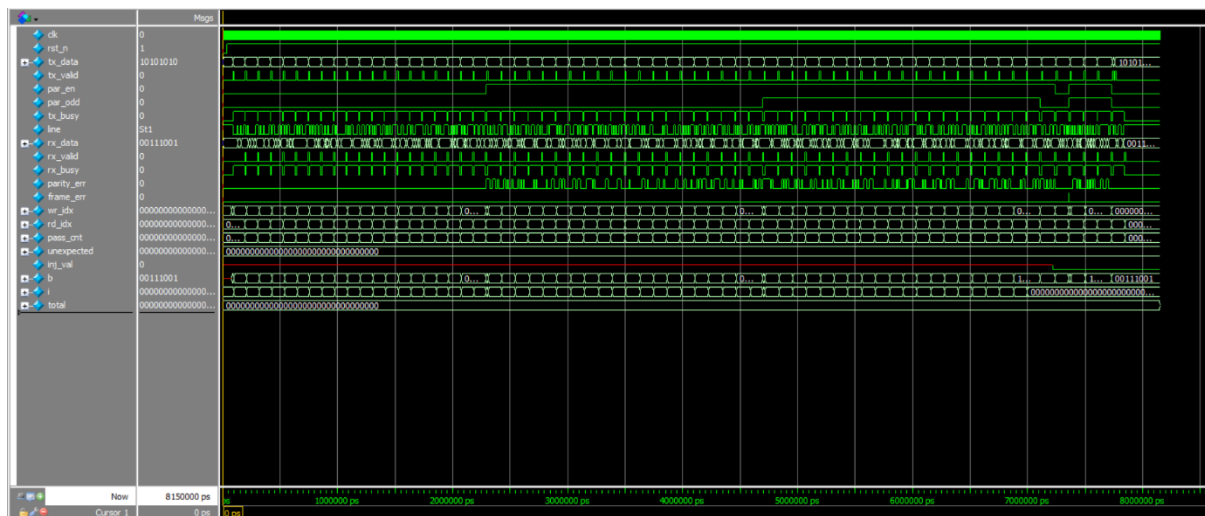
SIPO #(.DATA_W(DATA_W)) Sipo ( .in(i_rx), .clk(i_clk), .rst(i_rst_n), .en(SIPO_EN), .parallel(Data) );

assign o_data = Data;

endmodule

```

Simulation Waves:



The Testbench was provided from the instructor and this is the final Result:

```

# frames sent      : 66
# frames delivered : 66
# unexpected frames: 0
# SCORE 66 66
# GRADING: 66 / 66 frames correct
# RESULT: PASS
# ** Note: $finish   : D:/University/IC_Design/NTI/Labs/Final Final Project/Day 14/uart_loopback_tb.sv(279)
# Time: 8150 ns Iteration: 1 Instance: /uart_grading_tb
# 1
# Break in Module uart_grading_tb at D:/University/IC_Design/NTI/Labs/Final Final Project/Day 14/uart_loopback_tb.sv line 279

```

Full Result:

```

run -all
# === 1) random data, P_EN=0 ===
# ok[0]: d=0x24 pe=0 fe=0
# ok[1]: d=0x81 pe=0 fe=0
# ok[2]: d=0x09 pe=0 fe=0
# ok[3]: d=0x63 pe=0 fe=0
# ok[4]: d=0x0d pe=0 fe=0
# ok[5]: d=0x8d pe=0 fe=0
# ok[6]: d=0x65 pe=0 fe=0
# ok[7]: d=0x12 pe=0 fe=0
# ok[8]: d=0x01 pe=0 fe=0
# ok[9]: d=0x0d pe=0 fe=0
# ok[10]: d=0x76 pe=0 fe=0
# ok[11]: d=0x3d pe=0 fe=0
# ok[12]: d=0xed pe=0 fe=0
# ok[13]: d=0x8c pe=0 fe=0
# ok[14]: d=0xf9 pe=0 fe=0

```

```

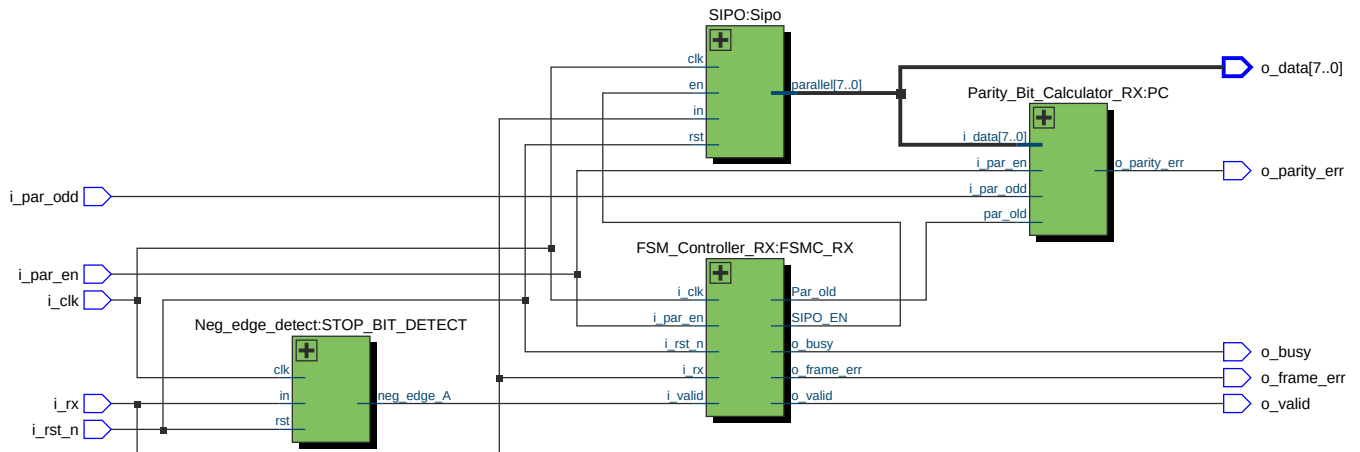
# ok[15]: d=0xc6 pe=0 fe=0
# ok[16]: d=0xc5 pe=0 fe=0
# ok[17]: d=0xaa pe=0 fe=0
# ok[18]: d=0xe5 pe=0 fe=0
# ok[19]: d=0x77 pe=0 fe=0
# === 2) random data, P_EN=1 P_BIT=0 (even) ===
# ok[20]: d=0x12 pe=0 fe=0
# ok[21]: d=0x8f pe=0 fe=0
# ok[22]: d=0xf2 pe=0 fe=0
# ok[23]: d=0xce pe=0 fe=0
# ok[24]: d=0xe8 pe=0 fe=0
# ok[25]: d=0xc5 pe=0 fe=0
# ok[26]: d=0x5c pe=0 fe=0
# ok[27]: d=0xbd pe=0 fe=0
# ok[28]: d=0x2d pe=0 fe=0
# ok[29]: d=0x65 pe=0 fe=0
# ok[30]: d=0x63 pe=0 fe=0
# ok[31]: d=0x0a pe=0 fe=0
# ok[32]: d=0x80 pe=0 fe=0
# ok[33]: d=0x20 pe=0 fe=0
# ok[34]: d=0xaa pe=0 fe=0
# ok[35]: d=0x9d pe=0 fe=0
# ok[36]: d=0x96 pe=0 fe=0
# ok[37]: d=0x13 pe=0 fe=0
# ok[38]: d=0x0d pe=0 fe=0
# ok[39]: d=0x53 pe=0 fe=0
# === 3) random data, P_EN=1 P_BIT=1 (odd) ===
# ok[40]: d=0x6b pe=0 fe=0
# ok[41]: d=0xd5 pe=0 fe=0
# ok[42]: d=0x02 pe=0 fe=0
# ok[43]: d=0xae pe=0 fe=0
# ok[44]: d=0x1d pe=0 fe=0
# ok[45]: d=0xcf pe=0 fe=0
# ok[46]: d=0x23 pe=0 fe=0
# ok[47]: d=0x0a pe=0 fe=0
# ok[48]: d=0xca pe=0 fe=0
# ok[49]: d=0x3c pe=0 fe=0
# ok[50]: d=0xf2 pe=0 fe=0
# ok[51]: d=0x8a pe=0 fe=0
# ok[52]: d=0x41 pe=0 fe=0
# ok[53]: d=0xd8 pe=0 fe=0
# ok[54]: d=0x78 pe=0 fe=0
# ok[55]: d=0x89 pe=0 fe=0
# ok[56]: d=0xeb pe=0 fe=0
# ok[57]: d=0xb6 pe=0 fe=0
# ok[58]: d=0xc6 pe=0 fe=0
# ok[59]: d=0xae pe=0 fe=0
# === 4) injected parity error ===
# ok[60]: d=0xc3 pe=1 fe=0
# === 5) injected framing error ===
# ok[61]: d=0x5c pe=0 fe=1
# === 6) back-to-back frames ===
# ok[62]: d=0xa5 pe=0 fe=0
# ok[63]: d=0x5a pe=0 fe=0
# ok[64]: d=0xff pe=0 fe=0
# === 7) busy-reject ===
# ok[65]: d=0x39 pe=0 fe=0
#
# frames sent      : 66
# frames delivered : 66
# unexpected frames: 0
# SCORE 66 66
# GRADING: 66 / 66 frames correct
# RESULT: PASS
# ** Note: $finish      : D:/University/IC_Design/NTI/Labs/Final Final Project/Day 14/uart_loopback_tb.sv(279)
#   Time: 8150 ns  Iteration: 1  Instance: /uart_grading_tb
# 1
# Break in Module uart_grading_tb at D:/University/IC_Design/NTI/Labs/Final Final Project/Day 14/uart_loopback_tb.sv
line 279

```

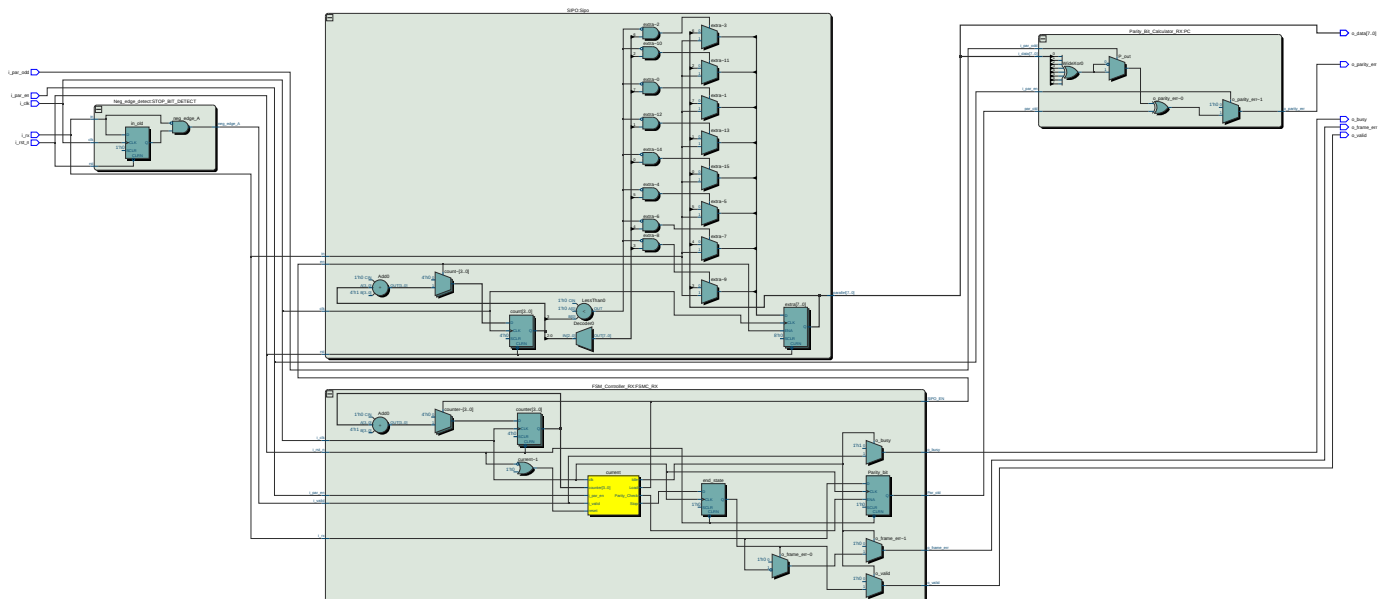
Schematics from Quartus Prime:

After we have discussed about the RX and learned about its process and implementation in System Verilog, here are some Schematics that relates to the RX.

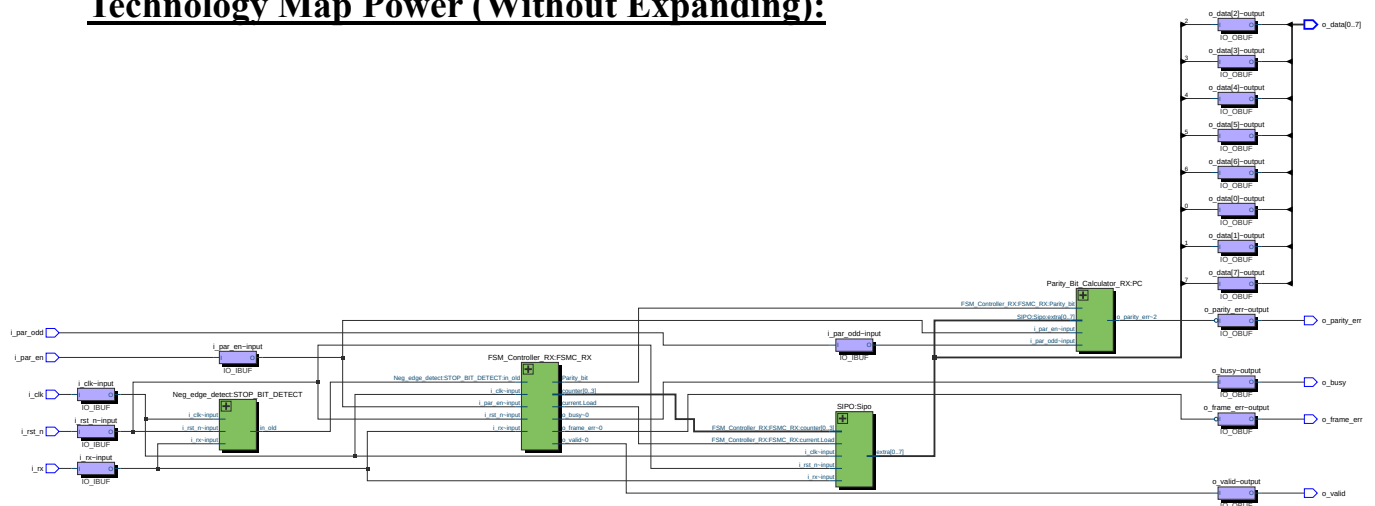
RTL VIEWER Schematic (Without Expanding):



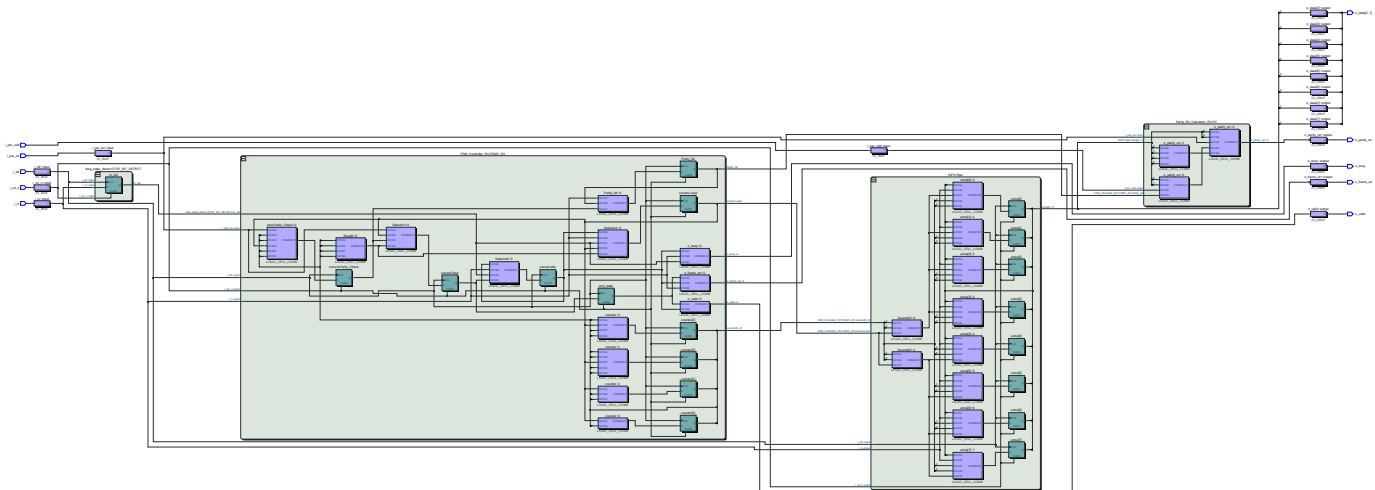
RTL VIEWER Schematic (With Expanding):



Technology Map Power (Without Expanding):



Technology Map Power (With Expanding):



Summary:

This project successfully implemented a fully functional UART Receiver. The design:

- ✓ Receives 10-bit or 11-bit serial data in standard UART frame format.
- ✓ Supports configurable Parity Check (Even / Odd / None).
- ✓ Passes all functional verification tests.

Future Improvements:

1. Implement a 32-bit FIFO (First In First Out) buffer for extra data.
2. Implement an APB.

And finally, I hope this project helped you understand how UART communication works and FPGA design, I am sure I did learn a lot, And I hope that I continue improving on this project!